Notes on
Foundations of Programming Languages

Mann Malviya
mannmalviya15@gmail.com

These notes provide an introduction to Foundations of Programming Languages. The primary sources used to write these notes were:

- CSE 114A: Foundations of Programming Languages — UCSC

Please report any errors to mannmalviya15@gmail.com

Contents

Lecture 1 **3**
What is PL? 3
What do PL people do? 3

Lecture 2 **5**

Lecture 21 **6**

Lecture 1:

What is PL?

For starters PL here doesn't mean programming languages as in the tools one uses to express computer programs(like C or python), it means programming languages the subfield of computer science. So what is the field of programming languages?

PL is one of the most misunderstood areas of computer science, it's even misunderstood by other computer scientists who are not specialists in PL. This is because when people hear "programming languages", they think of particular programming languages, languages they might have used or heard of. But the essence of the field is not about the details of this or that programming language.

What do PL people do?

- PL people consider the **programming language** to have a central place in solving computing problems.
- This might sound kind of obvious, "like of course we use programming languages to solve computing problems, what else would we do?" But this is actually an unusual and niche point of view, among computer scientists. A lot of computer scientists don't actually think the programming language that they might use to solve a given problem is all that important. They are concerned with the problem that they want to solve, but the question of what language they might use to implement a solution to that problem not so much, and if they think about it at all they think it's an implementation detail and not very interesting or important to the problem itself. They might figure that most languages are more or less the same any way, they are all Turing complete and close enough, so who cares?!
- The PL point of view is different and says that the programming language itself has a central place in the problem solving process, i.e., the choice of language matters and the choice of abstractions that a language offers also matters.
- So if you take the PL point of view, the way that your approach to solving a computing problem might as well be by first designing a programming language to solve that problem and then using that language you designed, to implement a solution!
- So to think about things as being a programming language design and implementation problem first, that's a PL way of approaching problem solving.
- This is not how every computer scientist goes about solving problems, this is a niche way of going about things — Turn everything into a language design and language implementation problem first & then go from there. At the extreme this might mean designing a new language to solve every new problem that you have!
- PL people consider software behavior in a rigorous and general way.
  - By rigorous we mean that computer programs are actually mathematical objects that we can reason about formally using the tools of math, in fact programs are actually things you can write mathematical proofs about!

- By general, we mean, if you are a PL person, you are often not concerned by just one particular program, you are going to be concerned by a whole class of programs, maybe all of the programs that are expressible in some particular programming language.
- PL people often make claims about **all** the programs that can be written using some particular programming language (later in the course we will make claims about **haskell** programs broadly)
- It's very powerful to be able to say something about all of the programs that are expressible within a particular programming language, or all of the programs that use or don't use a particular language Feature.
- Example: Rust is a memory safe language, which means that there's a whole class of memory related errors that cannot happen, they are completely ruled out when you are writing Rust (As long as you are not using the unsafe fragment of the language and barring language implementation issues).

Lecture 2:

Lecture 21:

We have spent a lot of time writing a lot of interpreters. Why do interpreters matter so much? No matter what code you're running, there's always an interpreter, at some level the CPU is an interpreter, it turns syntax of assembly language programs into semantics.

That's what an interpreter is, it turns syntax into semantics!

But interpreters are not the only thing that we are studying, in general we are studying programs that operate on programs.

Agenda:

- Intro to type checking and type inference
- A type system for "mini-nano"
- A tiny, terrible type inferencer (if time)

So we've built interpreters that take ASTs and produce values, and last time we extended our interpreter to function definitions & function calls. The language we are creating is starting to feel pretty real! But what happens if we create an AST that does something like applying a number to a number?

Looking at a snippet of the code from last time:

```
data Expr = Number Int

...
eval::Expr -> Env -> Value
```